Breaks — 2 Coffee breaks
- AM — 10:30 Am
- PM — 3:00 pm

15 mins

Lunch break — 12:30 pm IST — 1 hr

Yes = +1 , No = +2

"Github Copilot"? → "AI Coding Assistant" → AI Code mate → AI Code Engineer

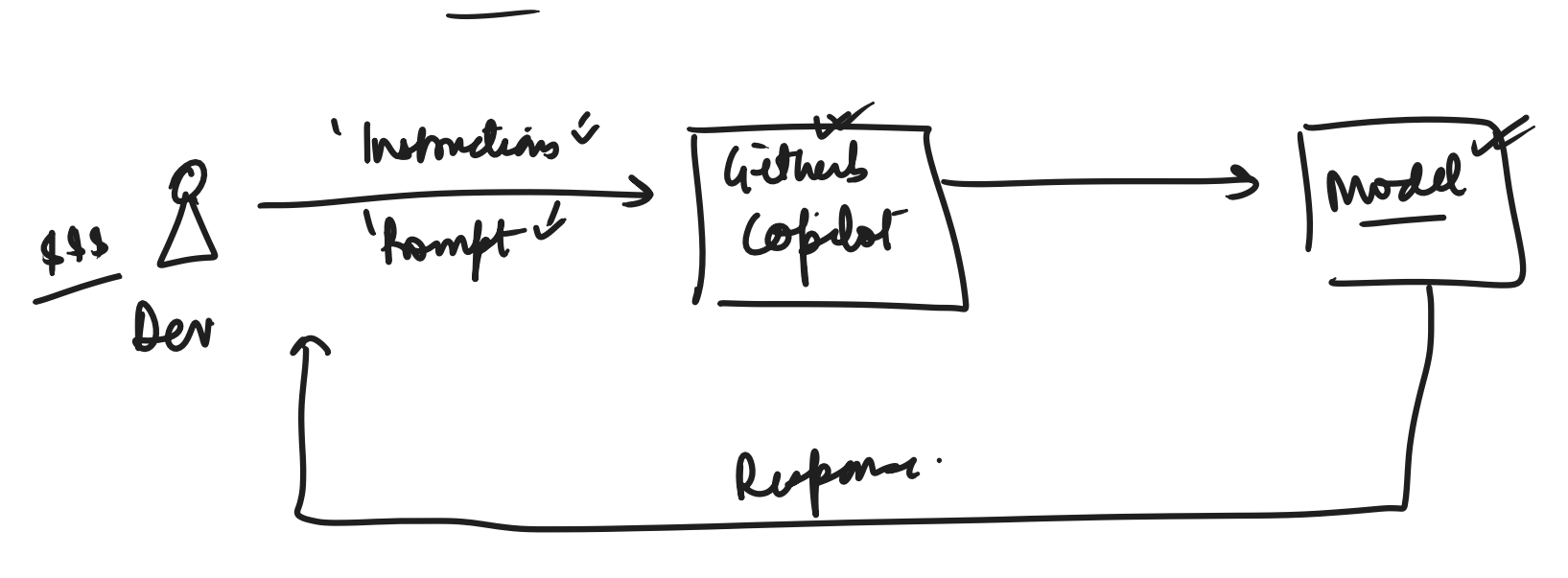
"Generative AI" → Create a new content – Text, Code, Image, video, Audio.

↳ Powered By "FM / LLM"

Foundation Model / LLM (Large Language Model) → NLP

↳ Subset of FM

Providers → AI21 — Jamba
- Anthropic — Claude
- Open AI — GPT
- Amazon — Nova



* FM / LLM * → Pretrained Model which are trained on Internet scale of data.
↳ Raw Data (Text, image, video ... etc)

↳ "General Domain Knowledge."

Model → Tokens
- Input
- output

Prompt → [Translate | this | into | french | → | 'I | love | cats' | .]

Input
↳ Tokens → numerical Rep (vector)

Model A → Input tokens = 12
→ output token = 3

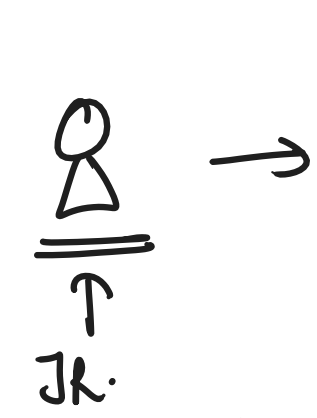
15

Interact with Copilot

1. Inline suggestions
2. Copilot Inline chat
3. Copilot chat

PROMPT Engineering

↳ Input , Instructions.



→ NEED Context + Rules + AC

1. Prompt Engineering for Copilot
2. Best Practices
3. Common mistakes + fixes.
4. Hands-on Lab.

Mental Model :

Copilot ⇒ Context + Rules + Target output

1. Context = existing code, file structure, interfaces, examples.
2. Rules / Constraints = language, style, lib, security + Performance rules.
3. Target output = function, test, Refactor, explanation, debug code.

PROMPT → Build an API

"Golden Prompt Template"

1. ✓ Goal = what you want
2. ✓ Input / output = function signature, data shape
3. ✓ Rules = Must / Must not
4. ✓ Edge cases : nulls, empty array, invalid input
5. ✗ Examples = one / two
6. Deliverables = code + Tests + explanation

Best Practices

1. Give Copilot "the world it lives in"
   → file, sample data, nearby type / interface
2. Ask for incremental output, not a whole system.
   1. generate type + func signature
   2. implement core logic
   3. add validation
      - Tests
      - refactor
3. Decision Transparency
   ✓ give 2 options & recommend one ff. with tradeoffs.
4. Control style & safety

Common Mistake

1. Vague Prompts
   X → Make this code better
   ✓ → Performance, complexity, edge cases,
2. Asking for too much at a once
   X → API + DB + Auth + test
   ✓ → Break it down
3. Not pinning dependencies / environment
   ** → Node version, TS strict mode, test runner
4. Copying code without verification

Pre-req
- Node.js installed (18+) *
- VS Code
- Github Copilot Chat ext + signed in

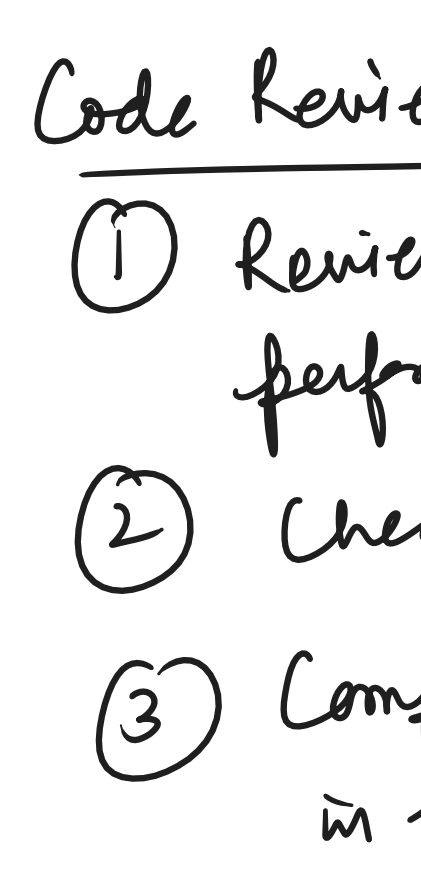
1. Initialise npm
   → npm init -y
2. Install test tooling (Vitest)
   → npm -i -D vitest

1. Prompt Engineering for Code Generation

Goal → Generate a Robust normalizeEmail (Email) & tests.

Copilot for Team Productivity

1. Pair programming with Copilot.
2. Copilot in Code review.



Copilot = Junior-to-mid Level Developer.

(✓)	(X)
– Starting a function from a clear comment	– Automatic source of truth.
– Repetitive code	– Replacement for design thinking
– Explain unfamiliar code.	– Security Reviewer
– Unit Tests	– Substitute of Domain knowledge
→ Refactor the code.	

Code Review Use Cases :

1. Review this function for the possible bugs related to performance.
2. Check / Identify missing test cases for this function
3. Compare this implementation with the existing coding style in this project.

Github Copilot → Control / guide it behavior.
1. ✓ Readme.md (Soft guide)
2. Copilot instructions (custom / Repo instruction)
   ↳ STRICT guide.

Readme.md → Copilot Reads
- ✓ (open files → context)
- ✓ Nearby files
- ✓ Repo context → Readme.md

- ↳ Project Architecture ✓✓
- ↳ Coding structure / Domain
- ↳ API Contracts
- ↳ folder structure ✓✓
- ↳ Naming conventions

} Context ground for code suggestions.

2. Use Copilot Instructions (STRONGER & More precise)

Where ? .github / copilot-instructions.md

What ? Github Supports
1. Repo-level instructions
2. org-level instructions.

When ? Need governance
Enterprise environment
Standardized output
TRG a team
Consistent Test generation